Standards Compliance Report

Web Conference Application

Assessment of 15 standards across WebRTC, RTP/SRTP, ICE, QUIC, HTTP/3, WebCodecs, and WebTransport

Application	web-conference
Assessment Date	March 2, 2026
Standards Assessed	15
Total Requirements	420
Directly Applicable	5 standards
Indirectly Applicable	3 standards
Not Applicable	7 standards

Overall Assessment: **Mostly Compliant** for applicable standards

Security posture: **Partially Compliant**

Generated from automated compliance analysis of source code against official W3C and IETF specifications.

Contents

1 Executive Summary **3**
- 1.1 Scope . . . 3
- 1.2 Key Findings . . . 3
- 1.3 Top Cross-Cutting Issues . . . 3

2 Application Overview **5**
- 2.1 Architecture . . . 5
- 2.2 Server Implementation . . . 5
- 2.3 Signaling Flow . . . 5
- 2.4 WebRTC Configuration . . . 6
- 2.5 Media Handling . . . 6
- 2.6 Connection State Management . . . 6
- 2.7 Network Topology . . . 7
- 2.8 Security Posture . . . 7
- 2.9 Dependencies . . . 7

3 Compliance Overview **8**
- 3.1 Directly Applicable Standards . . . 8
- 3.2 Indirectly Applicable Standards . . . 8
- 3.3 Not Applicable Standards . . . 8

4 Directly Applicable Standards — Detailed Findings **9**
- 4.1 W3C WebRTC 1.0: Real-Time Communication Between Browsers . . . 9
  - 4.1.1 Non-Compliant and Partially Compliant Findings . . . 9
  - 4.1.2 Recommendations . . . 10
- 4.2 W3C WebRTC 1.0 Editor's Draft . . . 11
  - 4.2.1 Non-Compliant and Partially Compliant Findings . . . 11
  - 4.2.2 Recommendations . . . 11
- 4.3 RFC 8829 — JavaScript Session Establishment Protocol (JSEP) . . . 13
  - 4.3.1 Non-Compliant and Partially Compliant Findings . . . 13
  - 4.3.2 Recommendations . . . 13
- 4.4 RFC 8826 — Security Considerations for WebRTC . . . 15
  - 4.4.1 Non-Compliant and Partially Compliant Findings . . . 15
  - 4.4.2 Recommendations . . . 15
- 4.5 RFC 8827 — WebRTC Security Architecture . . . 17
  - 4.5.1 Non-Compliant and Partially Compliant Findings . . . 17
  - 4.5.2 Recommendations . . . 17

5 Indirectly Applicable Standards — Detailed Findings **18**
- 5.1 RFC 8445 — Interactive Connectivity Establishment (ICE) . . . 18
  - 5.1.1 Non-Compliant and Partially Compliant Findings . . . 18
  - 5.1.2 Recommendations . . . 18
- 5.2 RFC 3550 — RTP: A Transport Protocol for Real-Time Applications . . . 19
  - 5.2.1 Partially Compliant Findings . . . 19
  - 5.2.2 Recommendations . . . 19
- 5.3 RFC 3711 — The Secure Real-time Transport Protocol (SRTP) . . . 19

5.3.1 Partially Compliant Findings . . . 19
5.3.2 Recommendations . . . 20

6 Not Applicable Standards **21**
6.1 Summary . . . 21
6.2 Future Considerations . . . 21

7 Consolidated Recommendations **22**
7.1 Critical Priority . . . 22
7.2 High Priority . . . 22
7.3 Medium Priority . . . 22
7.4 Low Priority . . . 23

8 Conclusion **24**

A Standards Reference **25**
A.1 WebRTC Standards . . . 25
A.1.1 W3C WebRTC 1.0: Real-Time Communication Between Browsers . . . 25
A.1.2 W3C WebRTC 1.0 Editor's Draft . . . 25
A.1.3 RFC 8829 — JavaScript Session Establishment Protocol (JSEP) . . . 26
A.1.4 RFC 8826 — Security Considerations for WebRTC . . . 26
A.1.5 RFC 8827 — WebRTC Security Architecture . . . 26
A.1.6 RFC 8445 — Interactive Connectivity Establishment (ICE) . . . 27
A.1.7 RFC 3550 — RTP: A Transport Protocol for Real-Time Applications . . . 27
A.1.8 RFC 3711 — The Secure Real-time Transport Protocol (SRTP) . . . 27
A.2 QUIC Standards . . . 28
A.2.1 RFC 9000 — QUIC: A UDP-Based Multiplexed and Secure Transport . . . 28
A.3 HTTP/3 Standards . . . 28
A.3.1 RFC 9114 — HTTP/3 . . . 28
A.4 WebCodecs Standards . . . 28
A.4.1 W3C WebCodecs . . . 29
A.5 WebTransport Standards . . . 29
A.5.1 RFC 9297 — HTTP Datagrams and the Capsule Protocol . . . 29
A.5.2 W3C WebTransport . . . 29

B Glossary **31**

Executive Summary

This report documents the compliance of the **web-conference** application against 15 industry standards governing real-time communication on the web. The application is a browser-based video conferencing tool built on WebRTC with Socket.IO signaling.

Scope

The assessment covers three categories of standards:

- **Directly Applicable (5 standards)** — Standards whose APIs and requirements the application directly implements: W3C WebRTC 1.0, W3C WebRTC Editor's Draft, RFC 8829 (JSEP), RFC 8826 (WebRTC Security), and RFC 8827 (WebRTC Security Architecture).
- **Indirectly Applicable (3 standards)** — Standards implemented by the browser where application configuration choices affect compliance: RFC 8445 (ICE), RFC 3550 (RTP), and RFC 3711 (SRTP).
- **Not Applicable (7 standards)** — Standards for technologies not used by the application: RFC 9000 (QUIC), RFC 9114 (HTTP/3), RFC 9297 (HTTP Datagrams), W3C WebCodecs (both drafts), and W3C WebTransport (both drafts).

Key Findings

Across the 8 applicable standards, the application was assessed against **178 requirements**, yielding:

Status	Count	Percentage
Compliant	88	49.4%
Partially Compliant	33	18.5%
Non-Compliant	25	14.0%
Not Applicable	26	14.6%
Not Assessable	6	3.4%

Top Cross-Cutting Issues

Six critical issues recur across multiple standards:

1. **No TURN servers** — Only Google STUN servers configured; connections fail behind symmetric NATs or restrictive firewalls. *Cited in 6 reports.*
2. **Wildcard CORS (`origin: '*'`)** — Undermines origin-based security model; any website can connect to the signaling server. *Cited in 4 reports.*
3. **No user authentication** — Anyone can join any room with a self-assigned username. *Cited in 2 reports (both critical severity).*
4. **No ICE restart** — Connection failures result in permanent peer removal with no recovery attempt. *Cited in 4 reports.*

5. **No `onnegotiationneeded` handler** — Cannot renegotiate sessions mid-call. *Cited in 2 reports.*
6. **Deprecated API usage** — Uses `new RTCSessionDescription()` and `new RTCIceCandidate()` constructors. *Cited in 3 reports.*

Application Overview

This section describes the current implementation of the web-conference application to provide context for the compliance findings that follow. Understanding the architecture and design decisions is essential for interpreting the gap analysis against each standard.

Architecture

The web-conference application is a self-hosted, peer-to-peer video conferencing system with four main components:

1. **Signaling Server** — A Node.js server using Express.js and Socket.IO that relays session negotiation messages between peers. It does not process or inspect media.
2. **Client Application** — A monolithic JavaScript class (`WebConference`, approximately 1,000 lines) that manages all WebRTC peer connections, media tracks, and UI interactions.
3. **Web Frontend** — HTML/CSS pages for the landing screen (room creation/joining) and the meeting room (video grid, controls, chat sidebar).
4. **AI Assistant** (optional) — An OpenAI-powered chatbot activated when an API key is configured, providing in-meeting conversational AI with text-to-speech.

Server Implementation

The server is built on Node.js with the following stack:

Component	Technology
HTTP Server	Node.js `http.createServer()`
Web Framework	Express.js 4.18
Real-Time Transport	Socket.IO 4.7 (WebSocket with long-polling fallback)
Static Assets	Express static middleware serving `/public`
AI Integration	OpenAI SDK 4.24 (optional)

The server maintains an in-memory `Map<roomId, Map<socketId, userInfo>>` for room and participant management. Rooms are created implicitly when the first user joins and destroyed when the last user leaves. All state is ephemeral — a server restart clears all rooms and sessions.

CORS configuration is set to `origin: '*'` with `GET` and `POST` methods allowed. The server listens on plain HTTP; HTTPS termination is delegated to the hosting platform (Render.com).

Signaling Flow

The application implements the JSEP (JavaScript Session Establishment Protocol, Appendix A.1.3) signaling pattern over Socket.IO:

1. A user joins a room; the server emits `existing-users` with the list of current participants and broadcasts `user-joined` to others.
2. For each existing peer, the new user and the existing peer each create an `RTCPeerConnection`. The existing peer acts as the **initiator** (offerer).

3. The initiator calls `createOffer()`, sets the local description, and sends the SDP offer via Socket.IO.
4. The answerer receives the offer, sets the remote description, calls `createAnswer()`, sets the local description, and sends the SDP answer back.
5. ICE candidates are trickled continuously — each peer sends candidates as they are discovered via the `onicecandidate` handler. Candidates arriving before the remote description is set are buffered in a `pendingCandidates` map and flushed after `setRemoteDescription` completes.

WebRTC Configuration

The `RTCPeerConnection` is constructed with the following ICE server configuration:

```
iceServers: [
    { urls: 'stun:stun.l.google.com:19302' },
    { urls: 'stun:stun1.l.google.com:19302' }
]
```

Only STUN servers are configured. No TURN servers are present, which means relayed candidates cannot be generated. This is the root cause of connectivity failures in restrictive network environments (see Sections 3–5 for related findings across multiple standards).

Default values are used for all other `RTCConfiguration` properties: `bundlePolicy: 'balanced'`, `rtcpMuxPolicy: 'require'`, `iceTransportPolicy: 'all'`, and `iceCandidatePoolSize: 0`.

Media Handling

Local media acquisition uses `getUserMedia({video: true, audio: true})` with a fallback to audio-only if the camera is unavailable. Screen sharing uses `getDisplayMedia({video: true})`.

Track management:

- Local audio and video tracks are added to each peer connection via `addTrack(track, stream)`.
- Screen sharing replaces the video track in all peer connections using `RTCRtpSender.replaceTrack()` — no renegotiation is required.
- Audio and video muting is implemented via `track.enabled = false`, which causes the browser to send black frames (video) or silence (audio) rather than stopping the RTP stream entirely.
- The `track.onended` event is monitored on screen share tracks to automatically revert to the camera when the user stops sharing via the browser's built-in controls.

Connection State Management

The application monitors `onconnectionstatechange` on each peer connection. When the state transitions to `disconnected` or `failed`, the peer is immediately removed: the connection is closed, video elements are removed from the DOM, and remaining participants are notified.

Notably, the application does **not**:

- Attempt ICE restart before removing a peer (see RFC 8445, Appendix A.1.6)

- Differentiate between `disconnected` (potentially transient) and `failed` (permanent)
- Monitor `oniceconnectionstatechange` for ICE-specific state transitions
- Handle the `onnegotiationneeded` event for mid-session renegotiation
- Implement glare handling for simultaneous offers (see RFC 8829, Appendix A.1.3)

Network Topology

The application uses a **full-mesh peer-to-peer** topology: each participant maintains a direct `RTCPeerConnection` to every other participant. For N participants, this requires $\frac{N(N-1)}{2}$ connections, making it practical for small groups (2–6 participants) but unsuitable for larger meetings without migration to an SFU (Selective Forwarding Unit) or MCU (Multipoint Control Unit) architecture.

Security Posture

The current security implementation is minimal:

Aspect	Status	Details
Authentication	None	Self-assigned usernames in `sessionStorage`
Authorization	None	Any user can join any room by ID
CORS	Open	`origin: '*'` on both Express and Socket.IO
Transport Encryption	Partial	DTLS-SRTP for media (browser-enforced); HTTP for signaling
HTTPS	Delegated	Render.com terminates TLS; server itself is HTTP
Room ID Entropy	Moderate	12 alphanumeric characters ($\approx 36^{12}$ combinations)

Media streams are encrypted end-to-end via DTLS-SRTP (Appendix A.1.8), which the browser enforces automatically. However, the signaling channel carries SDP containing DTLS fingerprints over an unauthenticated, wildcard-CORS HTTP connection, undermining the security guarantees defined by RFC 8826 (Appendix A.1.4) and RFC 8827 (Appendix A.1.5).

Dependencies

Package	Version	Purpose
`express`	^4.18.2	HTTP server and routing
`socket.io`	^4.7.2	WebSocket-based signaling
`dotenv`	^16.3.1	Environment variable management
`openai`	^4.24.0	AI assistant integration (optional)
`uuid`	^9.0.1	UUID generation utility

Compliance Overview

Directly Applicable Standards

Standard	Level	Compliant	Partial	Non-Compl.	N/A	Recs
W3C WebRTC 1.0	Mostly Compliant	19	6	5	5	9
W3C WebRTC 1.0 ED	Mostly Compliant	17	5	4	4	6
RFC 8829 JSEP	Mostly Compliant	20	6	4	5	6
RFC 8826 Security	Partially Compliant	4	4	5	2	6
RFC 8827 Sec. Arch.	Partially Compliant	6	4	3	5	5

Indirectly Applicable Standards

Standard	Level	Compliant	Partial	Non-Compl.	N/A	Recs
RFC 8445 ICE	Partially Compliant	7	4	4	3	4
RFC 3550 RTP	Mostly Compliant	8	2	0	4	3
RFC 3711 SRTP	Mostly Compliant	7	2	0	4	2

Not Applicable Standards

The following 7 standards were assessed and determined to be **not applicable**. The application does not use QUIC, HTTP/3, WebCodecs, or WebTransport. All 267 requirements across these standards were classified as not applicable.

Standard	Findings	Status
RFC 9000 — QUIC	45	Not Applicable
RFC 9114 — HTTP/3	40	Not Applicable
RFC 9297 — HTTP Datagrams	22	Not Applicable
W3C WebCodecs	30	Not Applicable
W3C WebCodecs Editor's Draft	30	Not Applicable
W3C WebTransport	30	Not Applicable
W3C WebTransport Editor's Draft	30	Not Applicable

Directly Applicable Standards — Detailed Findings

W3C WebRTC 1.0: Real-Time Communication Between Browsers

Standard ID: W3C-WebRTC-1.0
Source: `standards/webrtc/w3c-webrtc-1.0.html` (28,009 lines)
Applicability: Direct — uses RTCPeerConnection, getUserMedia, getDisplayMedia
Overall: Mostly Compliant
Requirements: 35 assessed (19 compliant, 6 partial, 5 non-compliant, 4 N/A, 1 N/A-able)

Non-Compliant and Partially Compliant Findings

ID	Section	Status	Severity	Description
F-002	4.2.2 RTCIceServer	Partial	Major	Only STUN servers configured; no TURN. Connections fail in restrictive NAT environments.
F-013	4.4.2 connectionState	Partial	Minor	No ICE restart on `failed`. No `closed` state handling. No `connected` state feedback to user.
F-014	4.7 negotiationneeded	Non-Compl.	Major	No `onnegotiationneeded` handler. Relies on explicit offer creation only at initial setup.
F-021	4.6.2 RTCSessionDescription	Partial	Minor	Uses deprecated `new RTCSessionDescription()` and `new RTCIceCandidate()` constructors.
F-022	4.5.1 Error Handling	Partial	Minor	Error handling limited to `console.error`. No user-facing notifications or retry logic.
F-023	4.3.3 PeerConnectionState	Partial	Minor	`disconnected` treated same as `failed` (immediate peer removal). No monitoring of `connecting`/`connected`.
F-024	4.3.4 IceConnectionState	Non-Compl.	Minor	No `oniceconnectionstatechange` handler.
F-028	8. getStats()	Non-Compl.	Minor	No `getStats()` usage for connection quality monitoring.
F-029	13.2 IP Addresses	Non-Compl.	Major	No IP address privacy consideration. Default `all` policy exposes host candidates. No TURN for relay-only mode.
F-030	13.4 Confidentiality	Partial	Major	CORS wildcard (`*`). Server uses HTTP not HTTPS. SDP transits through server without authentication.

ID	Section	Status	Severity	Description
F-032	4.4.2 restartIce()	Non-Compl.	Major	No ICE restart mechanism. Connection failures result in permanent peer removal.

Recommendations

1. HIGH **Implement ICE restart for connection recovery.** Use `restartIce()` or `createOffer({ iceRestart: true })` when state reaches `failed`. *Effort: Small. Findings: F-013, F-024, F-032.*

2. HIGH **Add TURN server configuration.** Only STUN is configured; TURN is essential for connectivity through symmetric NATs and firewalls. *Effort: Small. Findings: F-002, F-029.*

3. HIGH **Secure the signaling channel.** Restrict CORS from `*` to application domain, enforce HTTPS, add signaling authentication. *Effort: Medium. Findings: F-030.*

4. MEDIUM **Implement `onnegotiationneeded` handler.** Essential for session renegotiation and future extensibility. *Effort: Small. Findings: F-014.*

5. MEDIUM **Remove deprecated constructor usage.** Pass plain objects to `setRemoteDescription/addIc` *Effort: Trivial. Findings: F-021.*

6. MEDIUM **Improve connection state handling granularity.** Differentiate `disconnected` from `failed`; add timeout before removing peers; monitor ICE-specific states. *Effort: Small. Findings: F-013, F-023, F-024.*

7. LOW **Implement connection quality monitoring.** Use `getStats()` to monitor bytes sent/received, packet loss, and RTT. *Effort: Medium. Findings: F-028.*

8. LOW **Add IP address privacy controls.** Provide `iceTransportPolicy: 'relay'` option; requires TURN server. *Effort: Small. Findings: F-029.*

9. LOW **Enhance error handling with user notifications.** Show toast notifications for failures; implement retry logic. *Effort: Small. Findings: F-022.*

W3C WebRTC 1.0 Editor's Draft

Standard ID: W3C-WebRTC-1.0-Editors-Draft
Source: `standards/webrtc/w3c-webrtc-1.0-editors-draft.html` (18,374 lines)
Applicability: Direct — uses RTCPeerConnection API, getUserMedia, related WebRTC APIs
Overall: Mostly Compliant
Requirements: 30 assessed (17 compliant, 5 partial, 4 non-compliant, 3 N/A, 1 N/A-able)

Non-Compliant and Partially Compliant Findings

ID	Section	Status	Severity	Description
F-001	4.2.1 RTCConfiguration	Partial	Major	No TURN servers; connectivity fails behind symmetric NATs.
F-006	4.3.2 Unloading Cleanup	Partial	Minor	No `beforeunload` event handler for graceful connection cleanup when tab closes.
F-009	4.4.3 RTCSessionDescription	Partial	Minor	Uses deprecated `new RTCSessionDescription(...)` for `setRemoteDescription` calls.
F-012	4.4.2 addIceCandidate	Partial	Minor	Uses deprecated `new RTCIceCandidate(...)` in `addIceCandidate` calls.
F-015	4.4.3.3 connectionstatechange	Partial	Minor	No ICE restart on `failed`; immediate peer removal on `disconnected` may be premature.
F-026	4.4.3.5 negotiationneeded	Non-Compl.	Major	No `onnegotiationneeded` handler; manual offer management only at initial setup.
F-030	Security — HTTPS	Non-Compl.	Critical	Server uses plain HTTP (`http.createServer`). Relies entirely on Render.com for HTTPS termination.

Recommendations

1. CRITICAL **Add TURN server configuration.** Essential for reliable connectivity across all network types. *Effort: Small. Findings: F-001.*
2. HIGH **Implement `onnegotiationneeded` handler.** Required for renegotiation support; also handle glare scenarios. *Effort: Medium. Findings: F-026.*
3. HIGH **Implement ICE restart on connection failure.** Use `createOffer({ iceRestart: true })` when state reaches `failed`. *Effort: Small. Findings: F-015.*
4. MEDIUM **Remove deprecated constructor usage.** Pass plain objects to `setRemoteDescription` and `addIceCandidate`. *Effort: Trivial. Findings: F-009, F-012.*

5. MEDIUM **Add HTTPS support or document HTTPS requirements.** Add `http-to-https` redirect, configure HSTS. *Effort: Small. Findings: F-030.*

6. LOW **Add `beforeunload` handler for graceful cleanup.** Close peer connections when tab/window closes. *Effort: Trivial. Findings: F-006.*

RFC 8829 — JavaScript Session Establishment Protocol (JSEP)

Standard ID: RFC-8829
Source: `standards/webrtc/rfc8829-jsep.txt` (4,896 lines)
Applicability: Direct — implements JSEP signaling flow: offers/answers, SDP, ICE trickling
Overall: Mostly Compliant
Requirements: 35 assessed (20 compliant, 6 partial, 4 non-compliant, 4 N/A, 1 N/A-able)

Non-Compliant and Partially Compliant Findings

ID	Section	Status	Severity	Description
F-023	5.6 Processing Remote Desc	Partial	Major	No signaling state check or glare handling before `setRemoteDescription`.
F-024	5.2.3.1 IceRestart	Non-Compl.	Major	No ICE restart implemented; peer removed on failure instead of attempting recovery.
F-027	7.1 Simple Example	Partial	Minor	Answerer adds tracks before `setRemoteDescription` rather than after (works but deviates from JSEP example).
F-028	7.2 Trickle ICE Example	Partial	Minor	Initial trickle ICE correct, but no subsequent renegotiation supported.
F-029	4.1.10 RTCSessionDescription	Partial	Minor	Uses legacy explicit constructors instead of plain init objects.
F-033	5.2 Constructing Offer	Partial	Minor	`createOffer` never called with `iceRestart: true`.
F-035	8 Security Considerations	Non-Compl.	Critical	Signaling over plain HTTP; CORS `*`; SDP with DTLS fingerprints could be intercepted or modified.

Recommendations

1. CRITICAL **Secure the signaling channel.** Enforce HTTPS, restrict CORS to application domain, authenticate signaling connections. *Effort: Small. Findings: F-035.*

2. HIGH **Implement ICE restart for connection recovery.** Call `createOffer({ iceRestart: true })` on `failed` state. *Effort: Small. Findings: F-024, F-033.*

3. HIGH **Implement signaling glare handling.** Check `signalingState` before `setRemoteDescription`; implement tiebreaking for simultaneous offers. *Effort: Medium. Findings: F-023.*

4. MEDIUM **Support mid-session renegotiation.** Implement `onnegotiationneeded` handler. *Effort: Medium. Findings: F-028.*

5. MEDIUM **Remove legacy constructor usage.** Pass plain objects directly to WebRTC methods. *Effort: Trivial. Findings: F-029.*

6. LOW **Add TURN server support.** Ensures connectivity in restrictive network environments. *Effort: Small.*

RFC 8826 — Security Considerations for WebRTC

Standard ID: RFC-8826
Source: `standards/webrtc/rfc8826-webrtc-security.txt` (1,156 lines)
Applicability: Direct — defines WebRTC threat model; relates to signaling security, CORS, device permissi
Overall: Partially Compliant
Requirements: 15 assessed (4 compliant, 4 partial, 5 non-compliant, 2 N/A)

Non-Compliant and Partially Compliant Findings

ID	Section	Status	Severity	Description
F-002	4.1.3 Origin Security	Partial	Major	HTTPS via Render.com, but server itself uses `http.createServer` with no HTTPS enforcement or redirect.
F-003	4.1.4 Calling Page	Partial	Major	No native TLS, no HSTS header, no HTTP-to-HTTPS redirect.
F-006	4.2.4 IP Location	Non-Compl.	Major	No TURN servers, no `iceTransportPolicy` option, ICE begins immediately on room join.
F-008	4.3.1 Retrospective	Partial	Minor	DTLS provides forward secrecy, but no mechanism to detect signaling server MITM on fingerprints.
F-009	3.3 CORS/SOP	Non-Compl.	Critical	CORS `origin: '*'` allows any origin to connect to the signaling server.
F-010	4.1.2 Signaling Auth	Non-Compl.	Critical	No authentication whatsoever. Self-assigned usernames in `sessionStorage`. Anyone can join any room.
F-011	4.3.2 During-Call Attack	Non-Compl.	Major	No identity verification, no IdP integration, no SDP integrity verification, no fingerprint display.
F-013	4.1.1 Screen Sharing	Partial	Minor	Browser consent obtained, but no application-level warning about screen sharing risks.
F-014	3.2 SOP/Signaling	Non-Compl.	Critical	CORS `*` combined with no authentication creates an open signaling server exploitable by any website.

Recommendations

1. CRITICAL **Restrict CORS to trusted origins.** Change from `*` to actual application domain(s). *Effort: Trivial. Findings: F-009, F-014.*

2. CRITICAL **Implement user authentication and room access control.** Add room passwords, JWT tokens, or Socket.IO authentication middleware. *Effort: Medium. Findings: F-010.*

3. HIGH **Enforce HTTPS at application level.** Add HTTP-to-HTTPS redirect, HSTS header, `helmet.js` middleware. *Effort: Small. Findings: F-002, F-003.*

4. HIGH **Add IP address privacy controls with TURN support.** Deploy TURN server; offer relay-only mode. *Effort: Medium. Findings: F-006.*

5. MEDIUM **Add peer identity verification.** Display DTLS fingerprints in UI for manual verification. *Effort: Medium. Findings: F-008, F-011.*

6. LOW **Enhance screen sharing warnings.** Show application-level confirmation dialog before sharing. *Effort: Trivial. Findings: F-013.*

RFC 8827 — WebRTC Security Architecture

Standard ID: RFC-8827
Source: `standards/webrtc/rfc8827-dtls-webrtc.txt` (1,899 lines)
Applicability: Direct — defines security architecture: DTLS-SRTP, origin security, peer identity
Overall: Partially Compliant
Requirements: 18 assessed (6 compliant, 4 partial, 3 non-compliant, 3 N/A, 2 N/A-able)

Non-Compliant and Partially Compliant Findings

ID	Section	Status	Severity	Description
F-001	6.1 Origin/Web Security	Partial	Major	Server uses `http.createServer`; no HTTPS enforcement, no HSTS header.
F-008	6.4 IP Location Privacy	Non-Compl.	Major	No TURN servers, no TURN-only option, ICE starts immediately without user accept/decline.
F-013	6.5 Security UI	Non-Compl.	Major	No security inspector UI: no encryption indicator, no cipher information, no fingerprint display.
F-014	4.1/6.1 Signaling Security	Partial	Critical	CORS * allows any origin; no signaling authentication mechanism.
F-015	3/4.1 Trust Model	Non-Compl.	Critical	No user authentication; anyone can join any room with any self-assigned username.
F-018	9.2 Privacy	Partial	Minor	No privacy disclosure about IP exposure; no TURN support for IP address privacy.

Recommendations

1. CRITICAL **Restrict CORS and add signaling authentication.** Use JWT tokens; restrict CORS to application domain. *Effort: Medium. Findings: F-014, F-015.*

2. CRITICAL **Implement user authentication.** Room passwords, OAuth/SSO, or similar mechanism. *Effort: Large. Findings: F-015.*

3. HIGH **Add HTTPS enforcement.** HTTP-to-HTTPS redirect, HSTS header. *Effort: Small. Findings: F-001.*

4. HIGH **Configure TURN servers and IP privacy controls.** Deploy TURN; offer `iceTransportPolicy: 'relay'` option. *Effort: Medium. Findings: F-008, F-018.*

5. MEDIUM **Implement security characteristics UI.** Lock icon for encryption status, cipher suite display, DTLS fingerprint display. *Effort: Medium. Findings: F-013.*

Indirectly Applicable Standards — Detailed Findings

These standards are implemented by the browser's WebRTC stack. The application's configuration choices (ICE servers, connection handling, track management) affect compliance.

RFC 8445 — Interactive Connectivity Establishment (ICE)

Standard ID: RFC-8445
Source: `standards/webrtc/rfc8445-ice.txt` (5,603 lines)
Applicability: Indirect — ICE implemented by browser; app configuration affects compliance
Overall: Partially Compliant
Requirements: 18 assessed (7 compliant, 4 partial, 4 non-compliant, 2 N/A, 1 N/A-able)

Non-Compliant and Partially Compliant Findings

ID	Section	Status	Severity	Description
F-001	5.1.1 Gathering	Partial	Major	Only STUN configured; relayed (TURN) candidates impossible.
F-002	5.1.1.2 Relayed Candidates	Non-Compl.	Critical	No TURN server credentials or URLs; no mechanism to configure TURN.
F-003	9 ICE Restarts	Non-Compl.	Major	No ICE restart; `removePeer` called on `disconnected/failed` instead.
F-010	19.1 IP Address Privacy	Non-Compl.	Minor	No mechanism to restrict candidate types or hide IP addresses.
F-015	17.1 NAT/Firewall Types	Non-Compl.	Critical	No TURN means no connectivity through symmetric NATs or restrictive firewalls.
F-017	5.1.1.2 DNS Discovery	Partial	Minor	STUN servers hardcoded rather than DNS-discovered.
F-018	2.4/8 ICE Restart/Concluding	Partial	Major	Both `disconnected` and `failed` trigger immediate peer removal; no restart attempt.

Recommendations

1. CRITICAL **Add TURN server support.** Essential for real-world connectivity. Use coturn or managed TURN service. *Effort: Medium. Findings: F-001, F-002, F-015.*
2. HIGH **Implement ICE restart.** Use `restartIce()` with exponential backoff before removing peers. *Effort: Small. Findings: F-003, F-018.*
3. MEDIUM **Add IP address privacy controls.** Expose `iceTransportPolicy: 'relay'` toggle in settings. *Effort: Small. Findings: F-010.*
4. LOW **Make ICE server configuration dynamic.** Fetch server list from signaling server at session start. *Effort: Small. Findings: F-017.*

RFC 3550 — RTP: A Transport Protocol for Real-Time Applications

Standard ID: RFC-3550
Source: `standards/webrtc/rfc3550-rtp.txt` (5,827 lines)
Applicability: Indirect — browser implements RTP; app's use of tracks affects session construction
Overall: Mostly Compliant
Requirements: 14 assessed (8 compliant, 2 partial, 0 non-compliant, 4 N/A)

Partially Compliant Findings

ID	Section	Status	Severity	Description
F-013	6.2 Track Management	Partial	Minor	`replaceTrack` works correctly but has no error handling; no support for simultaneous camera + screen share.
F-014	6.2 Audio/Video Enable	Partial	Minor	`track.enabled = false` sends black frames/silence (wastes bandwidth vs. deactivating encoding).

Recommendations

1. MEDIUM **Optimize muted track bandwidth.** Use `RTCRtpSender.setParameters()` to deactivate encoding when tracks are muted. *Effort: Small. Findings: F-014.*
2. LOW **Add error handling for `replaceTrack`.** Handle failures gracefully with user notification. *Effort: Trivial. Findings: F-013.*
3. LOW **Consider `getStats()` for quality monitoring.** Monitor packet loss, jitter, and round-trip time. *Effort: Medium.*

RFC 3711 — The Secure Real-time Transport Protocol (SRTP)

Standard ID: RFC-3711
Source: `standards/webrtc/rfc3711-srtp.txt` (3,139 lines)
Applicability: Indirect — browser handles SRTP; app should not undermine encryption
Overall: Mostly Compliant
Requirements: 13 assessed (7 compliant, 2 partial, 0 non-compliant, 4 N/A)

Partially Compliant Findings

ID	Section	Status	Severity	Description
F-008	9.4 RTP Header Conf.	Partial	Minor	RTP headers exposed by design; application cannot change this behavior.

ID	Section	Status	Severity	Description
F-013	3/6.5 Encryption Enforce.	Partial	Minor	App relies on browser defaults; no explicit verification that DTLS-SRTP is active, no UI indicator.

Recommendations

1. MEDIUM **Add encryption status verification and UI indicator.** Use `getStats()` to verify DTLS-SRTP is active; display a lock icon in the UI. *Effort: Small. Findings: F-013.*

2. LOW **Document SRTP security properties for users.** Explain what is and is not encrypted. *Effort: Trivial. Findings: F-008.*

Not Applicable Standards

The following seven standards were assessed for completeness and found to be **not applicable** to the current application architecture. The application uses HTTP/1.1 with Express.js, WebSocket via Socket.IO, and WebRTC for media transport. It does not use QUIC, HTTP/3, WebCodecs, or WebTransport.

Summary

Standard	Findings	Rationale
RFC 9000 — QUIC	45	App uses TCP (Express/Socket.IO), not QUIC. Media uses WebRTC ICE/DTLS-SRTP over UDP.
RFC 9114 — HTTP/3	40	App uses HTTP/1.1. Note: HTTP/3 does not support 101 Switching Protocols used by Socket.IO WebSocket upgrade.
RFC 9297 — HTTP Datagrams	22	Infrastructure-level HTTP extension; app uses standard HTTP/1.1 GET/POST.
W3C WebCodecs	30	App uses WebRTC API with `<video>` elements; does not use VideoEncoder/Decoder or AudioEncoder/Decoder.
W3C WebCodecs (ED)	30	Same as above; Editor's Draft has substantively identical normative sections.
W3C WebTransport	30	App uses Socket.IO (WebSocket); no WebTransport API, no HTTP/3, no QUIC.
W3C WebTransport (ED)	30	Same as above; Editor's Draft tracks latest features.

Future Considerations

While not currently applicable, several of these technologies may become relevant as the application evolves:

- **HTTP/3 (QUIC)** — Could improve signaling latency with 0-RTT connection establishment and eliminate head-of-line blocking. However, Socket.IO's WebSocket upgrade mechanism is incompatible with HTTP/3, requiring architectural changes.
- **WebTransport** — Could replace Socket.IO for signaling with independent streams, unreliable datagrams (useful for ephemeral data like typing indicators), and built-in QUIC benefits. Browser support is maturing (Chrome/Edge available; Firefox/Safari in progress).
- **WebCodecs** — Could enable advanced video processing (background blur, virtual backgrounds, custom filters) and explicit codec control (AV1, hardware-accelerated H.264, custom bitrate management).

Consolidated Recommendations

The following table consolidates all unique recommendations across all applicable standards, ordered by priority. Duplicate recommendations that appear across multiple standards are merged.

Critical Priority

#	Recommendation	Description	Effort	Standards
1	Restrict CORS to trusted origins	Change `origin: '*'` to application domain(s)	Trivial	RFC 8826, 8827, 8829, WebRTC 1.0
2	Implement user authentication	Room passwords, JWT, OAuth, or Socket.IO middleware	Medium	RFC 8826, 8827
3	Add TURN server configuration	Deploy coturn or managed TURN for NAT traversal	Small	WebRTC 1.0, WebRTC ED, RFC 8445, 8826, 8827, 8829

High Priority

#	Recommendation	Description	Effort	Standards
4	Implement ICE restart	`restartIce()` with backoff on `failed` state	Small	WebRTC 1.0, WebRTC ED, RFC 8829, 8445
5	Enforce HTTPS at application level	HTTP-to-HTTPS redirect, HSTS, `helmet.js`	Small	RFC 8826, 8827, WebRTC ED
6	Implement `onnegotiationneeded`	Handle renegotiation and glare scenarios	Medium	WebRTC 1.0, WebRTC ED, RFC 8829
7	Implement signaling glare handling	Check `signalingState` before `setRemoteDescription`	Medium	RFC 8829
8	Add IP privacy controls with TURN	`iceTransportPolicy: 'relay'` option	Medium	RFC 8826, 8827, 8445

Medium Priority

#	Recommendation	Description	Effort	Standards
9	Remove deprecated constructors	Pass plain objects to WebRTC methods	Trivial	WebRTC 1.0, WebRTC ED, RFC 8829
10	Improve connection state handling	Differentiate `disconnected` from `failed`; add timeouts	Small	WebRTC 1.0, RFC 8445
11	Add security characteristics UI	Lock icon, cipher suite, fingerprint display	Medium	RFC 8827, 3711
12	Add peer identity verification	Display DTLS fingerprints for manual verification	Medium	RFC 8826
13	Optimize muted track bandwidth	Deactivate encoding via `setParameters()`	Small	RFC 3550

Low Priority

#	Recommendation	Description	Effort	Standards
14	Implement `getStats()` monitoring	Track packet loss, jitter, RTT	Medium	WebRTC 1.0, RFC 3550
15	Enhance error handling with notifications	Toast notifications, retry logic	Small	WebRTC 1.0
16	Add `beforeunload` cleanup handler	Close peer connections on tab close	Trivial	WebRTC ED
17	Make ICE servers dynamic	Fetch from signaling server at session start	Small	RFC 8445
18	Enhance screen sharing warnings	Application-level confirmation dialog	Trivial	RFC 8826
19	Document SRTP security properties	Explain encryption to end users	Trivial	RFC 3711
20	Add `replaceTrack` error handling	Graceful failure with user notification	Trivial	RFC 3550

Conclusion

The web-conference application demonstrates **mostly compliant** implementation of core WebRTC APIs (RTCPeerConnection, getUserMedia, offer/answer exchange, trickle ICE) with correct signaling flow and media track management. The browser's built-in RTP and SRTP implementations are properly leveraged without interference.

However, the application has significant gaps in three areas:

1. **Security** — The combination of wildcard CORS, no user authentication, no HTTPS enforcement, and unprotected signaling creates a critical security posture. Any website can connect to the signaling server, join rooms, and potentially intercept SDP containing DTLS fingerprints.

2. **Connectivity** — The absence of TURN servers means the application fails for users behind symmetric NATs or restrictive firewalls — a common scenario in corporate and mobile networks. Combined with no ICE restart capability, connection failures are permanent.

3. **Resilience** — No `onnegotiationneeded` handler, no glare handling, no ICE restart, and aggressive peer removal on `disconnected` state mean the application cannot recover from transient network issues or adapt to mid-session changes.

The three **critical-priority** recommendations (restrict CORS, add authentication, deploy TURN) would address the most impactful issues with relatively modest implementation effort. Implementing all high-priority items would bring the application to a **fully compliant** status for the directly applicable standards.

The seven not-applicable standards (QUIC, HTTP/3, WebCodecs, WebTransport) represent future technology directions that may become relevant as the application scales or adds advanced features such as custom video processing, improved signaling transport, or server-side recording.

— End of Main Report —

Standards Reference

This appendix provides a description of each standard assessed in this report. Standards are grouped by technology area, corresponding to the subdirectories in the `standards/` folder of the project repository. A total of 15 standards from the W3C and IETF were assessed, comprising over 122,000 lines of normative text.

WebRTC Standards

The WebRTC (Web Real-Time Communication) standards define the browser APIs and underlying protocols that enable peer-to-peer audio, video, and data communication directly between web browsers without requiring plugins or intermediary servers for media relay. The web-conference application is built directly on these standards.

W3C WebRTC 1.0: Real-Time Communication Between Browsers

Publisher:	World Wide Web Consortium (W3C)
Status:	W3C Recommendation (with Candidate Amendments)
Date:	2011–2025
Source File:	`standards/webrtc/w3c-webrtc-1.0.html` (28,009 lines)
Applicability:	Direct

This is the primary W3C specification defining the JavaScript APIs for WebRTC. It specifies the `RTCPeerConnection` interface (the central object for establishing peer-to-peer connections), the offer/answer negotiation model, ICE candidate handling, media track management (`addTrack`, `replaceTrack`, `getSenders`), the `RTCDataChannel` for arbitrary data, the statistics model (`getStats`), and the `RTCSessionDescription`/`RTCIceCandidate` types. It also defines the `MediaStream` and `getUserMedia`/`getDisplayMedia` APIs for local media capture.

The web-conference application directly uses `RTCPeerConnection`, `getUserMedia`, `getDisplayMedia`, `addTrack`, `replaceTrack`, `createOffer`/`createAnswer`, `setLocalDescription`/`setRemoteDescription`, and `addIceCandidate` from this specification.

W3C WebRTC 1.0 Editor's Draft

Publisher:	World Wide Web Consortium (W3C)
Status:	Editor's Draft (working document)
Source File:	`standards/webrtc/w3c-webrtc-1.0-editors-draft.html` (18,374 lines)
Applicability:	Direct

The Editor's Draft represents the most current working version of the WebRTC specification, incorporating proposed amendments and corrections not yet formally published as a W3C Recommendation. It tracks the same API surface as the official release but may include clarifications, deprecation notices (such as the deprecation of explicit `RTCSessionDescription` and `RTCIceCandidate` constructors), and requirements for secure contexts (HTTPS). Assessing against both the official release and the Editor's Draft ensures compliance with both current and forthcoming requirements.

RFC 8829 — JavaScript Session Establishment Protocol (JSEP)

Publisher:	Internet Engineering Task Force (IETF)
Status:	Proposed Standard
Date:	January 2021
Source File:	`standards/webrtc/rfc8829-jsep.txt` (4,896 lines)
Applicability:	Direct

JSEP describes how JavaScript applications control the signaling plane of a multimedia session via the `RTCPeerConnection` API. While WebRTC deliberately does not mandate a specific signaling protocol, JSEP defines the *interface* between the application and the browser: how to create offers and answers, how to apply session descriptions, how the SDP state machine operates, how ICE candidates are trickled, and how ICE restart works.

The web-conference application implements the JSEP offer/answer flow with trickle ICE over a custom Socket.IO signaling channel. The signaling protocol itself (Socket.IO events for `offer`, `answer`, `ice-candidate`) is application-defined, which JSEP explicitly permits.

RFC 8826 — Security Considerations for WebRTC

Publisher:	Internet Engineering Task Force (IETF)
Status:	Proposed Standard
Date:	January 2021
Source File:	`standards/webrtc/rfc8826-webrtc-security.txt` (1,156 lines)
Applicability:	Direct

This RFC defines the threat model for WebRTC applications, analyzing security considerations from the perspective of both the browser and the web application. It covers threats related to access to local devices (camera, microphone, screen), the origin-based security model (same-origin policy, CORS), communications consent verification (ensuring both parties agree to communicate), IP location privacy (the risk of exposing a user's IP address through ICE candidates), communications security (DTLS-SRTP encryption), and the security properties of the calling page (HTTPS requirements).

Many of this standard's requirements are *application-level* decisions rather than browser-level — for example, whether the signaling channel is authenticated, whether CORS is restricted, and whether TURN servers are provided for IP privacy. These are areas where the web-conference application has significant gaps.

RFC 8827 — WebRTC Security Architecture

Publisher:	Internet Engineering Task Force (IETF)
Status:	Proposed Standard
Date:	January 2021
Source File:	`standards/webrtc/rfc8827-dtls-webrtc.txt` (1,899 lines)
Applicability:	Direct

This companion RFC to RFC 8826 defines the overall security architecture for WebRTC. It establishes the trust model (what the browser trusts, what the application controls), specifies that DTLS-SRTP is mandatory for all media, defines requirements for origin and web security (HTTPS, secure contexts), device permissions (user consent for camera/microphone access), communications consent (ICE verification), IP location privacy (TURN relay support), and web-based peer authentication (the Identity Provider mechanism).

The standard is particularly relevant for evaluating the application's signaling security, authentication model, and whether the security architecture allows users to verify peer identity.

RFC 8445 — Interactive Connectivity Establishment (ICE)

Publisher:	Internet Engineering Task Force (IETF)
Status:	Proposed Standard (obsoletes RFC 5245)
Date:	July 2018
Source File:	`standards/webrtc/rfc8445-ice.txt` (5,603 lines)
Applicability:	Indirect

ICE is the protocol that enables WebRTC peers to discover the best network path for communication, even when one or both parties are behind Network Address Translators (NATs) or firewalls. ICE works by gathering "candidates" (potential transport addresses) of three types: **host** candidates (local IP addresses), **server-reflexive** candidates (public-facing addresses discovered via STUN), and **relayed** candidates (addresses allocated on a TURN server).

While ICE is implemented entirely by the browser, the application's configuration choices directly affect its operation: the ICE server list (STUN/TURN URLs and credentials), the `iceTransportPolicy` (whether to use all candidates or only relayed ones), and whether ICE restart is triggered on connection failure. The web-conference application's lack of TURN servers means relayed candidates are never available, causing connectivity failures in restrictive network environments.

RFC 3550 — RTP: A Transport Protocol for Real-Time Applications

Publisher:	Internet Engineering Task Force (IETF)
Status:	Standard
Date:	July 2003
Source File:	`standards/webrtc/rfc3550-rtp.txt` (5,827 lines)
Applicability:	Indirect

RTP provides end-to-end network transport functions for applications transmitting real-time data such as audio and video. It defines the packet format, sequence numbering, timestamping, delivery monitoring via RTCP (RTP Control Protocol), and participant identification via SSRCs (Synchronization Sources) and CNAMEs.

In WebRTC, RTP is the transport layer for all audio and video media. The browser constructs, sends, receives, and processes RTP packets; the application influences RTP behavior through its use of `addTrack` (which creates RTP streams), `replaceTrack` (which changes the media source without creating a new stream), and `track.enabled` (which controls whether media is actively sent).

RFC 3711 — The Secure Real-time Transport Protocol (SRTP)

Publisher:	Internet Engineering Task Force (IETF)
Status:	Proposed Standard
Date:	March 2004
Source File:	`standards/webrtc/rfc3711-srtp.txt` (3,139 lines)
Applicability:	Indirect

SRTP is a profile of RTP that adds confidentiality (encryption), message authentication, and replay protection. In WebRTC, SRTP is mandatory — the browser encrypts all RTP media using

keys exchanged via DTLS (Datagram Transport Layer Security). The default cipher suite is AES-128-CM with HMAC-SHA1-80 authentication.

The web-conference application does not interact with SRTP directly, but it should not undermine SRTP's guarantees. The application's compliance is assessed on whether it avoids introducing insecure alternatives (such as SDES key exchange), whether it provides users with verification that encryption is active, and whether the signaling channel adequately protects the DTLS fingerprints exchanged in SDP.

QUIC Standards

RFC 9000 — QUIC: A UDP-Based Multiplexed and Secure Transport

Publisher:	Internet Engineering Task Force (IETF)
Status:	Proposed Standard
Date:	May 2021
Source File:	`standards/quic/rfc9000-quic.txt` (8,485 lines)
Applicability:	Not applicable

QUIC is a UDP-based transport protocol that provides multiplexed streams, built-in TLS 1.3 encryption, low-latency connection establishment (including 0-RTT), and connection migration across network changes. It was originally developed by Google and standardized by the IETF as the transport layer for HTTP/3.

The web-conference application does not use QUIC. Its HTTP server uses TCP-based HTTP/1.1 (Express.js), its signaling uses WebSocket via Socket.IO, and its media transport uses WebRTC's ICE/DTLS-SRTP over UDP (which is a separate protocol stack from QUIC). QUIC could become relevant if the application migrates to HTTP/3 for signaling or adopts WebTransport.

HTTP/3 Standards

RFC 9114 — HTTP/3

Publisher:	Internet Engineering Task Force (IETF)
Status:	Proposed Standard
Date:	June 2022
Source File:	`standards/http3/rfc9114-http3.txt` (3,234 lines)
Applicability:	Not applicable

HTTP/3 maps HTTP semantics over the QUIC transport protocol, replacing TCP with QUIC's multiplexed streams to eliminate head-of-line blocking and enable faster connection establishment. It provides the same HTTP request/response semantics as HTTP/1.1 and HTTP/2 but with improved performance characteristics.

The web-conference application uses HTTP/1.1 via Express.js. A critical consideration for any future migration is that HTTP/3 does *not* support the 101 Switching Protocols mechanism that Socket.IO uses for WebSocket upgrade. Migrating to HTTP/3 would require replacing Socket.IO with an alternative signaling mechanism such as WebTransport or Server-Sent Events.

WebCodecs Standards

W3C WebCodecs

Publisher:	World Wide Web Consortium (W3C)
Status:	Working Draft
Date:	November 27, 2025
Source Files:	`standards/webcodecs/w3c-webcodecs.html` (12,870 lines)
	`standards/webcodecs/w3c-webcodecs-editors-draft.html` (12,859 lines)
Applicability:	Not applicable

WebCodecs provides low-level JavaScript interfaces to the browser's built-in audio and video codecs, enabling applications to encode and decode media frames directly. The API includes `VideoEncoder`, `VideoDecoder`, `AudioEncoder`, `AudioDecoder`, along with raw media types like `VideoFrame`, `AudioData`, and encoded chunk types. It also includes `ImageDecoder` for efficient image decoding.

WebCodecs is designed for use cases requiring fine-grained codec control: video processing pipelines (background blur, virtual backgrounds), custom bitrate management, server-side recording/transcoding, and Scalable Video Coding (SVC). The web-conference application uses the higher-level WebRTC API, which handles encoding/decoding transparently through the browser's media pipeline, making WebCodecs not applicable.

Both the official Working Draft and the Editor's Draft were assessed. The Editor's Draft has substantively identical normative sections, with minor editorial differences.

WebTransport Standards

RFC 9297 — HTTP Datagrams and the Capsule Protocol

Publisher:	Internet Engineering Task Force (IETF)
Status:	Proposed Standard
Date:	August 2022
Source File:	`standards/webtransport/rfc9297-webtransport-http3.txt` (717 lines)
Applicability:	Not applicable

This RFC defines HTTP Datagrams, a convention for conveying multiplexed, potentially unreliable datagrams inside HTTP connections, and the Capsule Protocol for HTTP/1.1 and HTTP/2 fallback. In HTTP/3, HTTP Datagrams can be sent unreliably using the QUIC DATAGRAM extension. This mechanism provides the foundation for WebTransport over HTTP/3.

The web-conference application uses standard HTTP/1.1 GET/POST requests and does not require HTTP-level datagram support.

W3C WebTransport

Publisher:	World Wide Web Consortium (W3C)
Status:	Working Draft
Date:	January 6, 2026
Source Files:	`standards/webtransport/w3c-webtransport.html` (7,533 lines)
	`standards/webtransport/w3c-webtransport-editors-draft.html` (7,621 lines)
Applicability:	Not applicable

WebTransport is a web API for bidirectional communication between a browser and a server, built on QUIC and HTTP/3. It offers capabilities beyond WebSocket: multiple independent

streams (avoiding head-of-line blocking), both reliable streams and unreliable datagrams, unidirectional streams, and 0-RTT connection establishment.

WebTransport could serve as a future replacement for Socket.IO in the web-conference application's signaling channel. Unreliable datagrams would be particularly suitable for time-sensitive, loss-tolerant data such as ICE candidate trickle messages or presence indicators. Browser support is maturing (available in Chrome, Edge, and Opera; in progress for Firefox and Safari).

Both the official Working Draft and the Editor's Draft were assessed. The Editor's Draft tracks the latest features including stream priority and enhanced connection statistics.

Glossary

Term	Definition
CORS	Cross-Origin Resource Sharing — an HTTP mechanism that allows a server to indicate which origins are permitted to access its resources.
DTLS	Datagram Transport Layer Security — provides TLS-equivalent security for datagram protocols (UDP). Used in WebRTC for key exchange before SRTP media encryption begins.
DTLS-SRTP	The combination of DTLS key exchange with SRTP media encryption, mandatory in WebRTC.
Full Mesh	A network topology where every participant maintains a direct connection to every other participant.
Glare	A condition where both peers simultaneously send SDP offers, causing a conflict in the JSEP state machine.
HSTS	HTTP Strict Transport Security — a response header that instructs browsers to only access the site over HTTPS.
ICE	Interactive Connectivity Establishment — the protocol used by WebRTC to discover the optimal network path between peers through NATs and firewalls.
ICE Restart	A mechanism to refresh ICE credentials and re-gather candidates without tearing down the entire peer connection.
JSEP	JavaScript Session Establishment Protocol — defines how JavaScript controls WebRTC session negotiation.
MCU	Multipoint Control Unit — a server that receives media from all participants, mixes/composes it, and sends a single combined stream to each participant.
NAT	Network Address Translator — a device that maps private IP addresses to public ones, complicating direct peer-to-peer connectivity.
QUIC	A UDP-based multiplexed and secure transport protocol with built-in TLS 1.3, developed as the transport for HTTP/3.
RTP	Real-time Transport Protocol — the standard for delivering audio and video over IP networks.
SDP	Session Description Protocol — a format for describing multimedia session parameters (codecs, transport addresses, encryption keys).
SFU	Selective Forwarding Unit — a server that receives media streams from participants and selectively forwards them to other participants without mixing.
SRTP	Secure Real-time Transport Protocol — RTP with added encryption, authentication, and replay protection.

Term	Definition
STUN	Session Traversal Utilities for NAT — a protocol that allows a client behind a NAT to discover its public IP address and port.
Symmetric NAT	A NAT type that maps each {internal IP, internal port, destination IP, destination port} tuple to a unique external port, making direct peer-to-peer connectivity impossible without a relay (TURN) server.
Trickle ICE	A mechanism where ICE candidates are sent to the remote peer as they are discovered, rather than waiting for all candidates to be gathered before signaling.
TURN	Traversal Using Relays around NAT — a protocol that provides a relay server for cases where direct peer-to-peer connectivity fails.